PARCOURS .NET POUR DÉVELOPPEURS JAVA · MODULES 3 ET 4 · CHALLENGE DE FIN DE PARCOURS

Challenge final — ReservSalles

Une API web sécurisée et son application mobile et desktop, en 3 heures

Modalité	**Individuelle**. Le formateur ne répond qu'aux problèmes d'**environnement** (émulateur, SDK, réseau), pas aux questions de code.
Durée	**3 h maximum** : socle noté sur 100 en **2 h 30** · bonus facultatif (jusqu'à +20 points) en **30 min**
Acquis évalués	Module 3 (ASP.NET Core : API REST, EF Core, validation, JWT, erreurs) et module 4 (.NET MAUI : XAML, MVVM, Shell, HTTP, desktop)
Version	**.NET 8** · VS Code · émulateur Pixel 7 API 34 · Windows 10/11
Ressources autorisées	Vos ateliers, vos supports de cours et la documentation Microsoft Learn. Pas de code d'un autre stagiaire. Outils d'IA générative : selon la consigne du formateur.
Livrable	Une archive `NOM_Prenom_ReservSalles.zip` (sans `bin` ni `obj`) contenant l'API, l'application, un `README.md` et des captures d'écran

> **Objectif**
> Montrer que vous savez, **seul et sur un sujet nouveau**, construire une petite API web sécurisée avec ASP.NET Core, puis l'application .NET MAUI qui la consomme sur **Android** et sur **Windows**.

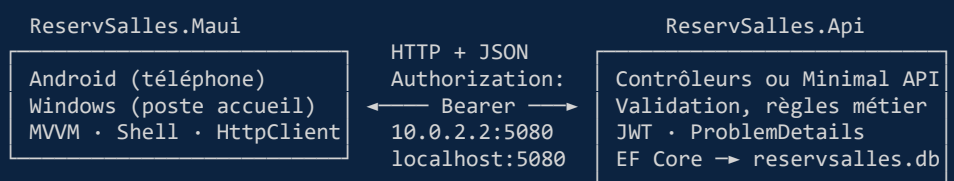
> **Livrable**
> Une API **ReservSalles.Api** conforme au contrat de ce document, une application **ReservSalles.Maui** qui fonctionne sur l'émulateur Android et sous Windows, et un README qui dit honnêtement ce qui fonctionne.

Contexte

Le centre de formation dispose de cinq salles de réunion. Aujourd'hui, les réservations se font sur un tableau papier à l'accueil : doubles réservations, salles trop petites, réservations oubliées. On vous demande un premier outil numérique :

- une **API web** qui gère les salles et les réservations, et refuse les réservations impossibles ;
- une **application** que les formateurs utilisent sur leur **téléphone** (réserver entre deux séances) et l'accueil sur son **poste Windows** (vue d'ensemble avec liste et détail côte à côte).

Architecture attendue



Règle du challenge

- Le sujet est **indépendant** du fil rouge GestionCommandes : nouveau domaine, nouveaux projets, nouvelle base.
- Il reprend pourtant **les mêmes techniques** : vous pouvez relire vos ateliers et réutiliser vos façons de faire (le gestionnaire qui ajoute le jeton, la configuration réseau Android, le maître-détail…). Il est en revanche interdit de copier le projet GestionCommandes et de le renommer.

Déroulement conseillé

Temps	Partie	Points
0:00 – 0:10	Lire tout le document, créer les deux projets	—
0:10 – 1:10	**Partie A** — API web (tâches A1 à A5)	40
1:10 – 2:10	**Partie B** — Application MAUI sur Android (tâches B1 à B5)	40
2:10 – 2:30	**Partie C** — Desktop, robustesse, livraison (tâches C1 à C3)	20
2:30 – 3:00	**Bonus** facultatif (D1 à D4, au choix), puis remise de l'archive	+20 max

Astuce

- **Avancez dans l'ordre** : l'application ne peut rien afficher sans l'API. Si une tâche vous bloque plus de 10 minutes, passez à la suivante et notez-le dans le README.
- Testez l'API **avant** d'écrire l'application, avec le fichier `.http` fourni plus bas : c'est le moyen le plus rapide de savoir si un problème vient du serveur ou du client.
- Une fonctionnalité partielle mais **honnêtement décrite** dans le README rapporte des points ; une fonctionnalité annoncée qui ne marche pas n'en rapporte pas.

Ce qui vous est imposé

Les projets

Élément	Valeur imposée
Dossier	`C:\Formation\Challenge` (avec un `global.json` en .NET 8, comme pour les ateliers)
API	`ReservSalles.Api` (modèle `webapi`), écoutant sur **http://localhost:5080** (profil `http`)
Base	SQLite, fichier `reservsalles.db`, créée au démarrage par EF Core (`EnsureCreated` ou migration, au choix)
Application	`ReservSalles.Maui`, cibles `net8.0-android` et `net8.0-windows10.0.19041.0`
Identifiant	`ApplicationId` = `com.<vos initiales>.reservsalles` (par exemple `com.mm.reservsalles`)

Les données de départ

Au démarrage de l'API, **si la base est vide**, insérez ces cinq salles :

Id	Nom	Capacité	Étage	Équipements
1	New York	12	1er étage	Écran, Visioconférence
2	Sydney	6	1er étage	Écran
3	Paris	20	2e étage	Vidéoprojecteur, Visioconférence
4	Tokyo	4	2e étage	Tableau blanc
5	Londres	40	Rez-de-chaussée	Vidéoprojecteur, Sonorisation

Ajoutez aussi deux réservations **pour le lendemain du jour de démarrage**, organisées par `formateur` : New York de 09:00 à 10:30 (« Réunion pédagogique », 8 participants) et Paris de 14:00 à 16:00 (« Jury de soutenance », 15 participants).

Deux comptes suffisent ; ils peuvent être lus dans `appsettings.json` (pas de table d'utilisateurs à créer) :

Identifiant	Mot de passe	Usage
`formateur`	`formateur123`	Organisateur des réservations de départ
`stagiaire`	`stagiaire123`	Second compte, pour tester les droits (règle R6)

Le contrat de l'API

L'application et la grille de correction s'appuient sur ce contrat : respectez **exactement** les routes, les noms de propriétés JSON et les codes de retour.

Méthode et route	Accès	Réponses attendues
`POST /api/auth/login`	Anonyme	200 `{ jeton, expiration }` · 401 si identifiants faux
`GET /api/salles?capaciteMin=10`	Anonyme	200 liste de salles, triée par nom ; `capaciteMin` facultatif
`GET /api/salles/{id}`	Anonyme	200 la salle · 404
`GET /api/salles/{id}/reservations?date=2026-10-20`	Jeton	200 réservations de ce jour, triées par heure de début · 404 si la salle n'existe pas
`POST /api/reservations`	Jeton	201 + en-tête `Location` · 400 · 404 (salle) · 409 (conflit)
`GET /api/reservations/miennes`	Jeton	200 réservations **à venir** de l'utilisateur connecté, triées
`DELETE /api/reservations/{id}`	Jeton	204 · 403 si l'utilisateur n'est pas l'organisateur · 404

Formats JSON (dates locales, sans fuseau horaire)

```
// SalleDto
{ "id": 1, "nom": "New York", "capacite": 12, "etage": "1er étage",
  "equipements": "Écran, Visioconférence" }

// CreerReservationDto (corps de POST /api/reservations)
{ "salleId": 1, "objet": "Point d'avancement", "participants": 5,
  "debut": "2026-10-20T11:00:00", "fin": "2026-10-20T12:00:00" }

// ReservationDto
{ "id": 7, "salleId": 1, "salleNom": "New York", "objet": "Point d'avancement",
  "participants": 5, "debut": "2026-10-20T11:00:00", "fin": "2026-10-20T12:00:00",
  "organisateur": "stagiaire" }
```

Les règles métier

Règle	Énoncé	Si elle n'est pas respectée
R1	L'objet est obligatoire, de 3 à 80 caractères	400, message sur le champ `objet`
R2	La fin est après le début, le même jour, et la réservation dure **4 h au plus**	400
R3	Le début est dans le futur	400
R4	Le nombre de participants est compris entre 1 et la **capacité de la salle**	400, message citant la capacité
R5	Une salle ne peut pas avoir deux réservations qui se chevauchent	**409**, message lisible (« La salle New York est déjà réservée de 09:00 à 10:30. »)
R6	Seul l'organisateur peut annuler sa réservation	**403**
R7	L'organisateur est l'utilisateur **du jeton**, jamais une valeur envoyée par le client	—

Astuce

Deux créneaux `[d1, f1[` et `[d2, f2[` se chevauchent si et seulement si `d1 < f2 && d2 < f1`. Une réservation qui commence exactement à la fin d'une autre (10:30 → 10:30) est donc **acceptée**.

⚠ Attention

Toutes les erreurs (400, 403, 404, 409) sont renvoyées au format **ProblemDetails** (`title`, `status`, `detail`, et `errors` pour la validation), afin que l'application puisse afficher le message à l'utilisateur.

Le fichier de test fourni

Créez `ReservSalles.Api/ReservSalles.http` avec ce contenu. Il se lance depuis VS Code (extension **REST Client**, ou le support `.http` du C# Dev Kit) : chaque requête affiche le code attendu en commentaire.

ReservSalles.Api/ReservSalles.http

```
@api = http://localhost:5080
@demain = 2026-10-20

### 1. Salles d'au moins 10 places → 200 (Londres, New York, Paris)
GET {{api}}/api/salles?capaciteMin=10

### 2. Salle inconnue → 404
GET {{api}}/api/salles/99
```

```
### 3. Connexion → 200 ; copiez le jeton dans @jeton
POST {{api}}/api/auth/login
Content-Type: application/json

{ "identifiant": "stagiaire", "motDePasse": "stagiaire123" }

###
@jeton = collez-ici-le-jeton

### 4. Sans jeton → 401
GET {{api}}/api/reservations/miennes

### 5. Réservation valide → 201
POST {{api}}/api/reservations
Authorization: Bearer {{jeton}}
Content-Type: application/json

{ "salleId": 1, "objet": "Point d'avancement", "participants": 5,
  "debut": "{{demain}}T10:30:00", "fin": "{{demain}}T11:30:00" }

### 6. Chevauchement avec la réunion de 09:00-10:30 → 409
POST {{api}}/api/reservations
Authorization: Bearer {{jeton}}
Content-Type: application/json

{ "salleId": 1, "objet": "Conflit", "participants": 2,
  "debut": "{{demain}}T10:00:00", "fin": "{{demain}}T10:45:00" }

### 7. Trop de participants pour Tokyo (4 places) → 400
POST {{api}}/api/reservations
Authorization: Bearer {{jeton}}
Content-Type: application/json

{ "salleId": 4, "objet": "Atelier", "participants": 9,
  "debut": "{{demain}}T15:00:00", "fin": "{{demain}}T16:00:00" }

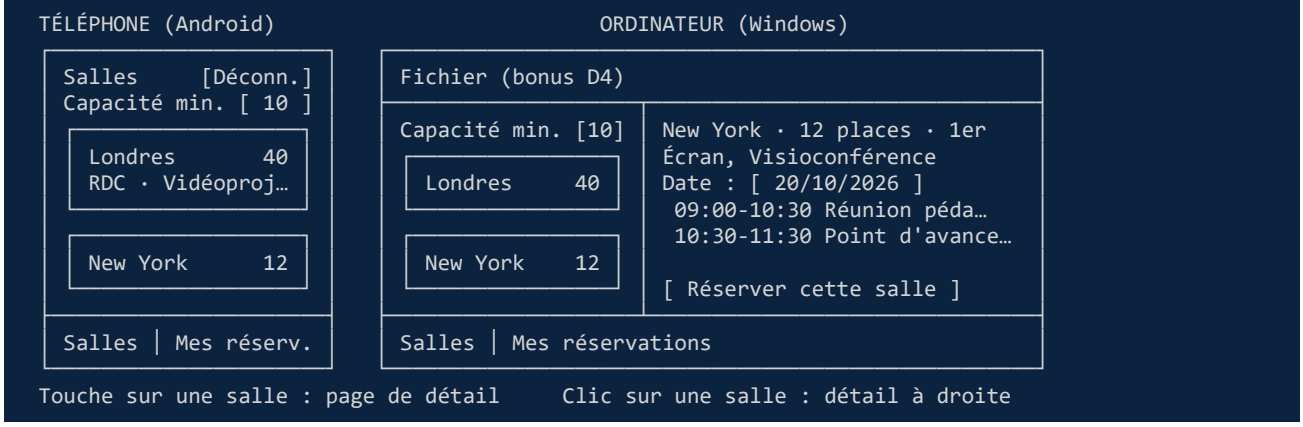
### 8. Mes réservations → 200 (la réservation créée en 5)
GET {{api}}/api/reservations/miennes
Authorization: Bearer {{jeton}}

### 9. Annuler la réservation de « formateur » (id 1) → 403
DELETE {{api}}/api/reservations/1
Authorization: Bearer {{jeton}}
```

> **⚠ Attention**
>
> Adaptez `@demain` à la date réelle du lendemain avant de lancer les requêtes.

Les écrans de l'application

Maquettes (indicatives)



Écran	Contenu minimal
Connexion	Identifiant, mot de passe, bouton **Se connecter**, message d'erreur ; hors des onglets
Salles (onglet 1)	Filtre « capacité minimale », liste des salles (nom, capacité, étage, équipements), indicateur de chargement
Détail d'une salle	Informations de la salle, sélecteur de date, réservations de ce jour, bouton **Réserver cette salle**
Nouvelle réservation	Objet, date, heure de début, heure de fin, participants ; **Enregistrer** grisé tant que la saisie est invalide ; message de l'API en cas de refus
Mes réservations (onglet 2)	Réservations à venir de l'utilisateur, **Annuler** avec confirmation, bouton **Déconnexion**

Partie A — L'API ReservSalles.Api

40 PTS · 1 H

Vous êtes libre du style (**contrôleurs** ou **Minimal API**) et de l'organisation interne, tant que le contrat et les règles sont respectés.

A1 — Projet, modèle et base · 8 pts · 12 min

- Entités `Salle` et `Reservation` (une salle a plusieurs réservations), `DbContext` enregistré dans l'injection de dépendances avec SQLite.
- Création de la base et insertion des données de départ au démarrage, uniquement si la base est vide.
- Swagger accessible sur `http://localhost:5080/swagger`.

☐ La base `reservsalles.db` est créée au premier démarrage

☐ Un second démarrage ne duplique pas les données

A2 — Lecture des salles · 8 pts · 10 min

- `GET /api/salles` (filtre `capaciteMin` facultatif, tri par nom) et `GET /api/salles/{id}`.
- Des **DTO** sont renvoyés, pas les entités EF Core (pas de boucle de sérialisation, pas de champ inutile).

☐ Requêtes 1 et 2 du fichier `.http` : 200 avec trois salles, puis 404

A3 — Authentification par jeton JWT · 8 pts · 12 min

- `POST /api/auth/login` vérifie le compte et renvoie un jeton signé contenant le **nom** de l'utilisateur (durée de validité : 2 h).
- Clé de signature, émetteur et audience lus dans la **configuration**, jamais écrits en dur dans le code.
- Toutes les routes « Jeton » du contrat sont protégées ; Swagger propose le bouton **Authorize**.

☐ Requête 3 : 200 avec un jeton ; mauvais mot de passe : 401

☐ Requête 4 : 401 sans jeton

A4 — Créer une réservation · 10 pts · 16 min

- `POST /api/reservations` applique les règles **R1 à R5** et **R7**, et répond 201 avec l'en-tête `Location`.
- `GET /api/salles/{id}/reservations?date=…` renvoie les réservations du jour, triées.

☐ Requête 5 : 201

☐ Requête 6 : 409 avec un message lisible

☐ Requête 7 : 400 citant la capacité de 4 places

☐ Une réservation de 10:30 à 11:30, juste après celle de 09:00-10:30, est acceptée

A5 — Mes réservations, annulation et erreurs · 6 pts · 10 min

- `GET /api/reservations/miennes` et `DELETE /api/reservations/{id}` (règle R6).
- Erreurs au format **ProblemDetails** ; aucune exception non gérée ne renvoie de page d'erreur ou de trace de pile.
- Une ligne de **journal** (`ILogger`) à chaque réservation créée ou annulée.

☐ Requête 8 : 200 ; requête 9 : 403

- ☐ Annuler sa propre réservation : 204, puis 404 si on recommence

> **⚠ Attention**
>
> - **Rappel des ateliers** : `app.UseHttpsRedirection()` doit être réservé à la production (`if (!app.Environment.IsDevelopment())`). Sinon, l'application reçoit une redirection qui fait **perdre l'en-tête `Authorization`** (401 incompréhensibles) ou un refus de certificat sur Android.
> - Lancez l'API avec `dotnet run --launch-profile http` et vérifiez dans sa console : `Now listening on: http://localhost:5080` et `Hosting environment: Development`.

Partie B — L'application ReservSalles.Maui sur Android

40 PTS · 1 H

Architecture **MVVM** obligatoire avec le **CommunityToolkit.Mvvm** : pas de logique dans le code-behind des pages, pas d'appel HTTP depuis une page, services et ViewModels obtenus par **injection de dépendances**.

B1 — Projet, navigation et connexion · 10 pts · 15 min

- `AppShell` : une page de connexion, puis une barre d'onglets **Salles** / **Mes réservations** ; routes enregistrées pour le détail et la saisie.
- Client HTTP typé, adresse de base selon la plateforme (`10.0.2.2` sur Android, `localhost` sous Windows), HTTP autorisé vers `10.0.2.2` seulement.
- Après connexion, le jeton est rangé dans `SecureStorage` et **ajouté automatiquement** à chaque requête par un gestionnaire HTTP.

- ☐ Mauvais mot de passe : message dans la page
- ☐ Bon mot de passe : arrivée sur l'onglet Salles

B2 — Liste des salles · 8 pts · 12 min

- `ObservableCollection` alimentée par une commande asynchrone, indicateur d'activité pendant le chargement.
- Filtre « capacité minimale » (`Entry` numérique ou `Stepper`) qui relance la recherche.
- Une salle par carte : nom, capacité, étage, équipements.

- ☐ Les cinq salles s'affichent
- ☐ Capacité minimale 10 : trois salles

B3 — Détail d'une salle · 6 pts · 10 min

- Navigation vers le détail avec l'**identifiant** de la salle en paramètre (`[QueryProperty]` ou `IQueryAttributable`).
- `DatePicker` (demain par défaut) : changer de date recharge les réservations du jour.

- ☐ New York, demain : la « Réunion pédagogique » de 09:00 à 10:30 apparaît

B4 — Nouvelle réservation · 10 pts · 15 min

- Formulaire objet, date, heures de début et de fin (`TimePicker`), participants.
- **Enregistrer** n'est actif que si la saisie est valide côté client (objet de 3 caractères au moins, fin après le début, participants ≥ 1) : `CanExecute` et notification.
- Réponse 400 ou 409 : le **message de l'API** (`detail` ou `errors`) est affiché dans la page ; succès : retour au détail, liste à jour.

- ☐ Conflit avec 09:00-10:30 : le message de l'API s'affiche, l'utilisateur reste sur le formulaire
- ☐ Réservation valide : visible au retour sur le détail

B5 — Mes réservations et déconnexion · 6 pts · 8 min

- Onglet **Mes réservations**, rechargé à chaque affichage de la page.
- **Annuler** (bouton, `SwipeView` ou menu contextuel) après confirmation (`DisplayAlert`) ; la liste se met à jour.

- **Déconnexion** : jeton effacé, retour à la page de connexion.

☐ Annuler sa réservation : elle disparaît

☐ Après déconnexion, l'onglet Salles n'est plus accessible sans se reconnecter

Partie C — Desktop, robustesse et livraison — 20 PTS · 20 MIN

C1 — Une vraie application de bureau · 8 pts · 8 min

- Sous Windows, la page **Salles** affiche la **liste à gauche et le détail à droite** (`OnIdiom` ou équivalent) ; sur téléphone, le comportement de B3 est conservé.
- Fenêtre ouverte en 1100 × 750, avec une taille minimale de 800 × 600.

C2 — Robustesse · 8 pts · 6 min

- API arrêtée : message « Serveur injoignable » dans la page, **aucun plantage**.
- Jeton refusé (401) : retour automatique à la page de connexion.

C3 — Livraison · 4 pts · 6 min

- `README.md` à la racine : comment lancer l'API et l'application, comptes de test, **ce qui fonctionne et ce qui ne fonctionne pas**.
- Dossier `captures/` : au moins une capture Android (formulaire de réservation) et une capture Windows (maître-détail).
- Archive sans `bin/` ni `obj/`, et sans la base `reservsalles.db`.

Bonus — 30 minutes au plus, jusqu'à +20 points

Uniquement si le socle est terminé. Chaque bonus est indépendant : choisissez ceux que vous pouvez finir dans le temps restant.

Bonus	Attendu	Points
D1 — Tests	Projet xUnit : au moins deux tests sur la règle de chevauchement (R5), ou deux tests d'un ViewModel avec des doubles de test	+5
D2 — Hors ligne	Les salles sont mises en cache dans SQLite (`sqlite-net-pcl`) et affichées quand l'API est injoignable, avec un bandeau « Hors ligne »	+5
D3 — Package	Un APK signé (Release) installé sur l'émulateur, ou un MSIX installé sous Windows, en version 1.0.0 ; commande utilisée notée dans le README	+5
D4 — Menu et raccourcis	Barre de menus **Fichier** sur la page Salles, sous Windows : **Actualiser** (Ctrl+R), **Nouvelle réservation** (Ctrl+N, sur la salle sélectionnée), **Quitter**	+5

> **Astuce**
> Pour D4, rappel de l'atelier A8 : les éléments de la barre de menus n'héritent pas de façon fiable du `BindingContext`. Nommez la page (`x:Name`) et liez les commandes par `Source={x:Reference …}, Path=BindingContext.…`.

Scénario de recette (utilisé pour la correction)

Le formateur déroule ce scénario sur votre livrable. Déroulez-le vous-même avant de rendre l'archive.

N°	Action	Résultat attendu	Tâche
1	Supprimer `reservsalles.db`, lancer l'API, ouvrir Swagger	Base recréée, 5 salles, 2 réservations demain	A1
2	Exécuter les requêtes 1 à 9 du fichier `.http`	200, 404, 200, 401, 201, 409, 400, 200, 403	A2–A5
3	Android : se connecter avec un mauvais puis un bon mot de passe	Message d'erreur, puis onglet Salles	B1
4	Filtrer sur 10 places	Londres, New York, Paris	B2
5	Ouvrir New York, choisir demain	Réunion pédagogique 09:00-10:30	B3
6	Réserver New York demain de 10:00 à 11:00	Message de conflit de l'API, formulaire conservé	B4
7	Réserver New York demain de 11:00 à 12:00	Retour au détail, nouvelle réservation visible	B4
8	Mes réservations : annuler celle de 11:00	Confirmation, puis disparition	B5
9	Windows : clic sur une salle	Détail à droite, fenêtre de 1100 × 750	C1
10	Arrêter l'API, actualiser	« Serveur injoignable », pas de plantage	C2
11	Ouvrir le README et le dossier captures	Lancement, comptes, état honnête, 2 captures	C3
12	Bonus : tests, mode hors ligne, package, puis sous Windows Ctrl+R et Ctrl+N	Tests verts ; salles affichées API arrêtée ; package installé ; liste rechargée, formulaire ouvert	D1–D4

Grille d'évaluation

Partie	Critère	Points
A — API	A1 Projet, modèle EF Core, base et données de départ	8
	A2 Lecture des salles, filtre, DTO, 404	8
	A3 Connexion JWT, configuration externalisée, routes protégées	8
	A4 Création : règles R1 à R5 et R7, codes 201 / 400 / 409	10
	A5 Mes réservations, annulation (R6), ProblemDetails, journalisation	6
B — MAUI	B1 Shell, client HTTP typé, connexion, jeton automatique	10
	B2 Liste des salles en MVVM, filtre, indicateur	8
	B3 Détail avec paramètre de navigation et sélection de date	6
	B4 Saisie validée (`CanExecute`), messages 400 / 409 de l'API	10
	B5 Mes réservations, annulation confirmée, déconnexion	6
C — Desktop	C1 Maître-détail sous Windows, taille de fenêtre	8
	C2 Robustesse : serveur injoignable, 401	8
	C3 README honnête, captures, archive propre	4
Total		**100**
Bonus	D1 Tests · D2 Hors ligne · D3 Package · D4 Menu et raccourcis (5 chacun, note plafonnée à 100)	+20

Note	Appréciation
Moins de 50	**Non validé** : un entretien de rattrapage est proposé

Note	Appréciation
50 à 69	**Validé** : les acquis essentiels des modules 3 et 4 sont maîtrisés
70 à 84	**Validé — bien** : API et application complètes, quelques manques sur le desktop ou la robustesse
85 et plus	**Validé — très bien** : autonomie confirmée sur une chaîne API + application multiplateforme

Règles de notation

- Un critère est accordé en entier s'il fonctionne lors de la recette, à moitié s'il est implémenté mais incomplet (et signalé dans le README), et pas du tout s'il ne compile pas ou n'est pas présent.
- Une API qui ne démarre pas rend le reste invérifiable : **faites-la démarrer en priorité**.

Avant de rendre l'archive

- ☐ L'API démarre sur http://localhost:5080 et les 9 requêtes du fichier .http donnent les codes attendus
- ☐ L'application se connecte et fonctionne sur l'émulateur Android
- ☐ L'application fonctionne sous Windows, avec la liste et le détail côte à côte
- ☐ Le README dit comment lancer, avec quels comptes, et ce qui ne fonctionne pas
- ☐ Deux captures dans captures/ ; ni bin/, ni obj/, ni reservsalles.db dans l'archive

Pour le formateur

Note pour le formateur

- **Avant la séance** : postes vérifiés (script `Verifier-Env.ps1` du guide d'installation), émulateur Pixel 7 API 34 démarré une première fois, port **5080** libre (`netstat -ano | findstr 5080`), dossier `C:\Formation\Challenge` vide avec son `global.json`.
- **Pendant** : n'intervenir que sur l'environnement (émulateur bloqué, SDK, pare-feu) et noter le temps perdu, à rendre au stagiaire. Annoncer l'heure à 1:10, 2:10 et 2:30.
- **Correction** (10 à 15 minutes par stagiaire) : décompresser dans un dossier neutre, `dotnet run --launch-profile http`, exécuter le fichier `.http`, puis dérouler le scénario de recette sur Android et Windows. Les numéros du scénario renvoient aux lignes de la grille.
- **Points d'attention** : organisateur pris dans le corps de la requête au lieu du jeton (R7, A4 incomplet) ; entités EF renvoyées directement (A2 incomplet) ; clé JWT écrite en dur (A3 incomplet) ; logique dans le code-behind (B2 à B5 incomplets) ; `Enregistrer` toujours actif (B4 incomplet).
- **Indépendance du fil rouge** : un projet GestionCommandes renommé (classes `Client`, `Commande` restées dans le code) est hors sujet.
- Les dates de départ sont calculées par rapport au jour de démarrage de l'API : faire corriger `@demain` dans le fichier `.http` avant la recette.